

Smriti Sathi PPT Technical Breakdown

Architecture, Stack, Data Flow, Code Snippets & Slides Outline

Smriti Sathi: Technical Breakdown for SIH Presentation

This document contains a comprehensive, technically accurate analysis of the Smriti Sathi codebase. It is specifically designed to help you build your Smart India Hackathon (SIH) presentation and defend your technical choices in front of the judges. ---

1. Technology Stack

- **Frontend Framework:** React 18.3.1 with TypeScript 5.6.3
- **Build Tool:** Vite 5.4.11
- **Mobile Shell (Cross-platform App):** Capacitor 8.5.0 (`@capacitor/android` , `@capacitor/ios`) for native deployment.
- **Database/Storage:** Offline-first IndexedDB (using raw IndexedDB API wrapped in native Promises).
- **AI/ML (On-Device):** Custom mathematical implementations in pure TypeScript using `Float32Array` and `Float64Array` (Multi-Layer Perceptron and LinUCB).
- **AI/ML (Cloud API):** Groq API for high-speed, low-latency LLM inference (utilizing `openai/gpt-oss-120b` , `openai/gpt-oss-20b` , and `qwen/qwen3.6-27b`).
- **Testing:** Vitest for unit testing, Playwright for multi-device End-to-End (E2E) testing.

2. System Architecture

Smriti Sathi uses an **Offline-First Edge AI Architecture** designed for zero-connectivity environments, with graceful enhancement when online.

- **Edge Tier (Core Engine):** Gameplay mechanics, clinical safety guardrails, user progress tracking, and the adaptive difficulty engine (LinUCB + MLP) run entirely on the local device's CPU.
- **Data Tier (Local Store):** All state, patient profiles, and analytical logs are saved locally in IndexedDB. There is no central cloud server required for core functionality, ensuring 100% data privacy and availability.
- **Cloud Tier (Enhancements):** When internet is available, the app reaches out to the Groq API for dynamic LLM-based caregiver summaries and chatbot interactions, providing empathetic, localized responses.

3. Project Structure

- `src/lib/` : Core brain of the application. Contains database logic (`db.ts`), game definitions (`games.ts`), AI integrations (`ai/`), and all mathematical/adaptive algorithms (`adaptive/` , `linucb.ts` , `srt.ts` , `sathi.ts`).
- `src/screens/` : React components acting as full-page views (e.g., `CaregiverHub.tsx`).
- `scripts/` : Build and automation scripts (e.g., PDF generators, icon generation).
- `ios/` & `android/` : Native Capacitor project folders for mobile compilation.

4. Frontend

- **Framework:** React 18 using functional components and hooks.
- **State Management:** Decentralized state using React Hooks, directly interfacing with the IndexedDB wrappers for persistence.
- **Routing:** Likely handled by a lightweight router or conditional rendering based on state, given the App/Screen structure.
- **Styling:** Designed for Gerontechnology Ergonomics—minimum 48×48px (96px primary) touch targets, WCAG 2.1 AAA contrast, using standard CSS/inline-styles.

5. Backend

- **Architecture: Serverless / Offline-First.** There is no traditional Node.js/Python backend server running a REST API.
- **Services:** The "backend" logic (business logic, AI difficulty adjustment, data persistence) is shipped within the frontend bundle and runs directly in the user's browser/mobile runtime context via TypeScript modules.

6. Database

- **Technology:** IndexedDB API (Browser native NoSQL).
- **Schema & Collections (Stores):**
 - * `kv` : Key-value store for singletons (`profile` , `config` , `bandit` , `srt`).
 - * `events` : Append-only ledger for analytics and gameplay events.
 - * `sessions` : Historical gameplay session records.
 - * `meds` & `medlog` : Medication management and tracking.
 - * `daily_reminders` & `appointments` : Scheduling.
- **Data Flow:** React components call async wrapper functions (e.g., `saveProfile` , `addSession`) which open an IndexedDB transaction and resolve a Promise upon success.

7. Core Features

- **Adaptive Cognitive Games (21 modules):** Games are dynamically selected and their difficulty is modulated based on past performance using a contextual bandit algorithm.
- **Spaced Retrieval Therapy (SRT):** Implements an expanding interval progression ($G = [1, 2, 4, 8]$) to reinforce memory retention.
- **Caregiver Hub:** Securely gated (4-digit PIN) dashboard showing patient statistics, medication schedules, and AI-generated progress summaries.

- **Dementia-Friendly Chatbot ("Sathi"):** A conversational AI companion tailored with patient context (language, stage) to provide gentle, localized interactions.

8. AI/ML Implementation

- **On-Device Multi-Layer Perceptron (MLP):** A lightweight neural network (\$25 \to 16 \to 8 \to 1\$) running entirely on device. It takes a 25-dimensional feature vector (accuracy, latency, streaks, regional relevance) and outputs a difficulty appropriateness score.
- **LinUCB Contextual Bandit:** An reinforcement learning algorithm that selects the best game to play next by balancing exploitation (known good games) and exploration (new games), updated in real-time using Sherman-Morrison rank-1 matrix updates.
- **Cloud LLMs:** Groq API integration using targeted system prompts with strict JSON output validation to generate caretaker summaries and adaptive game recommendations.

9. Security

- **Authentication:** Local PIN-based gatekeeping for the Caregiver Hub to prevent accidental access by the patient.
- **Data Privacy:** 100% on-device storage. Patient data never leaves the device unless explicitly queried to the Groq API (which is stateless).
- **API Security:** Groq API keys are utilized, likely injected via environment variables (`ENV_CONFIG`).

10. Data Flow (Example: Completing a Game)

1. **UI:** User finishes a game module. 2. **Processing:** The app calculates Accuracy, Latency (ms), and Frustration Index (tap burstiness). 3. **Adaptive Update:** The data is fed into the LinUCB algorithm (`shermanMorrisonUpdate`) to update the mathematical model of the user's cognitive state. 4. **Database:** The `SessionRecord` and `LedgerEvent` are written to IndexedDB. 5. **UI:** The screen transitions to the next dynamically selected activity.

11. Algorithms & Important Logic

- **Frustration Index (\$FI\$):** Calculates user frustration based on tap burstiness (e.g., multiple screen taps within $\Delta t < 180$ ms) and hesitation cues.
- **Cognitive Stability Index (CSI):** A dual-window moving average looking for sequence failures to provide early warnings for delirium or infections.
- **Clinical Safety Guardrails:** Deterministic rules preventing rapid difficulty jumps or error spirals, ensuring the patient experiences ~75% errorless learning (1PL Rasch Model).

12. APIs & Integrations

- **Groq API:** Primary external integration. Used for high-speed text completion (`/v1/chat/completions`). Features graceful degradation—if the API fails or is offline, the app seamlessly falls back to localized, pre-programmed responses.
- **Capacitor Plugins:** Bridges web APIs to native iOS/Android hardware capabilities.

13. Performance & Scalability

- **Memory Efficiency:** The on-device MLP is built using typed arrays (`Float32Array`), resulting in a model footprint of ~2.3 KB and an inference time of `<0.05 ms` on pure CPU (suitable for low-end devices like Android Go).
- **Scalability:** Because compute and storage are offloaded to the edge (the user's device), the infrastructure costs are virtually zero, and the system scales infinitely with the user base.

14. Testing

- **Frameworks:** Vitest (Unit) and Playwright (E2E).
- **Coverage:** E2E testing spans Desktop (Chromium, Firefox, Safari) and Mobile profiles (iPhone 14, iPad Pro 11, Pixel 7) to ensure responsive design integrity.

15. Deployment/Infrastructure

- **Build Pipeline:** Vite compiles the TypeScript and React code into optimized static assets.
- **Mobile Deployment:** Capacitor synchronizes these web assets into native Xcode and Android Studio projects for compilation into `.ipa` and `.apk / .aab` binaries.
- **Hosting:** As a static PWA/Web App, it can be hosted on any static CDN (Vercel, Netlify, GitHub Pages) with zero backend infrastructure.

16. Technical Challenges & Solutions

- **Challenge:** Running ML algorithms on low-end smartphones without internet access.
- **Solution:** Built a custom, ultra-lightweight MLP using `Float32Array` in pure TypeScript, bypassing heavy libraries like TensorFlow.js to achieve sub-millisecond execution locally.
- **Challenge:** Maintaining clinical safety while using AI for game selection.
- **Solution:** Implemented deterministic 5-Layer Clinical Safety Guardrails that override AI decisions if they violate errorless learning principles.

Key Code Snippets (For PPT)

Snippet 1: On-Device MLP Engine (Edge AI)

File: `src/lib/adaptive/MLInferenceEngine.ts` **Why it's important:** Demonstrates how you achieved offline machine learning. Instead of importing heavy libraries, you used raw memory buffers (`Float32Array`) to create a 2.3KB neural network that runs in `<0.05ms`.

```
/
* Ultra-Lightweight On-Device Multi-Layer Perceptron (MLP)
* Target Execution: Pure CPU, <0.05ms inference time, 100% offline.
*/
class MLInferenceEngineClass {
```

```

private W1: Float32Array
private b1: Float32Array
// ... (W2, b2, W3, b3 omitted for brevity)

constructor() {
  this.W1 = new Float32Array(16 * FEATURE_DIMENSION) // 25 → 16
  this.b1 = new Float32Array(16)
  // Memory efficient allocation for weights & biases
}

public predict(features: number[]): number {
  // Layer 1: 25 → 16 (Using ReLU activation)
  const h1 = new Float32Array(16)
  for (let i = 0; i < 16; i++) {
    let sum = this.b1[i]
    const rowOffset = i * FEATURE_DIMENSION
    for (let j = 0; j < FEATURE_DIMENSION; j++) {
      sum += this.W1[rowOffset + j] * features[j]
    }
    h1[i] = Math.max(0, sum) // ReLU
  }
  // ... passes through Layer 2 and Output Layer (Sigmoid)
}
}

```

Snippet 2: LinUCB Contextual Bandit (Reinforcement Learning)

File: `src/lib/linucb.ts` **Why it's important:** Shows the mathematical rigor of the app. This is the Sherman-Morrison rank-1 update equation implemented in TypeScript, allowing the app to learn patient preferences in real-time without retraining a model.

```

export function shermanMorrisonUpdate(arm: { Ainv: number[][]; b: number[]; n: number },
  // Ainv is the inverse covariance matrix, x is the context vector, r is the reward
  const Ax = matVec(arm.Ainv, x)
  const denom = 1 + dot(x, Ax)

  // Rank-1 update of the inverse covariance matrix
  for (let i = 0; i < arm.Ainv.length; i++) {
    for (let j = 0; j < arm.Ainv[i].length; j++) {
      arm.Ainv[i][j] -= (Ax[i] * Ax[j]) / denom
    }
  }

  // Update reward vector
  for (let i = 0; i < arm.b.length; i++) arm.b[i] += x[i] * r
  arm.n += 1
}

```

Snippet 3: Offline-First Database Engine

File: `src/lib/db.ts` **Why it's important:** Proves the application functions entirely offline. You wrote a custom Promise-wrapper around the native browser IndexedDB API to ensure fast, local,

asynchronous data storage.

```
function tx(store: string, mode: IDBTransactionMode, fn: (s: IDBObjectStore) => IDBRequest) => IDBRequest {
  return openDb().then(
    (db) =>
      new Promise((resolve, reject) => {
        const t = db.transaction(store, mode)
        const req = fn(t.objectStore(store))
        req.onsuccess = () => resolve(req.result)
        req.onerror = () => reject(new Error([db, store].join(' ') + ' error'))
      })
  )
}

// Highly efficient, local, NoSQL data interactions
export const dbGet = (store: string, key: IDBValidKey) => tx(store, 'readonly', (s) => s.get(key))
export const dbSet = (store: string, value: unknown, key?: IDBValidKey) =>
  tx(store, 'readwrite', (s) => {
    if (s.keyPath === null && key !== undefined) return s.put(value, key) as IDBRequest
    return s.put(value) as IDBRequest
  })
```

Snippet 4: Intelligent Prompting & JSON Validation

File: `src/lib/ai/AIService.ts` **Why it's important:** Demonstrates advanced API integration. You aren't just sending text to Groq; you are providing contextual prompt engineering and strictly validating the JSON output to generate automated caregiver insights.

```
const prompt = `Generate a non-diagnostic, respectful summary for a family caregiver of a patient with the following
PERFORMANCE DATA:
- Total sessions: ${sessions.length}
- Medication adherence rate: ${Math.round(adherencePct)}%

STRICT RULES:
1. DO NOT diagnose or claim to treat dementia or medical conditions.
2. Use cautious, uplifting, and clear language.
3. Respond in STRICT JSON matching:
{
  "headline": "string",
  "strengths": ["string", "string"],
  "observations": ["string", "string"],
  "suggestedFocus": "string"
}`

const res = await this.callGroq(
  [
    { role: 'system', content: 'You output valid JSON only.' },
    { role: 'user', content: prompt },
  ],
  { temperature: 0.2, topP: 0.95, maxTokens: 1024 }
```

```
{ model: this.primaryModel, temperature: 0.4, responseFormat: 'json_object' }  
)
```